Take Home Activity 2

Team 24

Adapter Pattern – Flight Reservation System

1. Pattern Description

The **Adapter Pattern** is a structural design pattern that allows objects with incompatible interfaces to collaborate. It acts as a wrapper between two different interfaces, translating calls from one interface into calls that the other can understand. The adapter converts the interface of an existing class (the *Adaptee*) into another interface (the *Target*) that the client expects, without modifying either side.

Key Participants

Role	Description
Target	The interface the client code expects
Adaptee	An existing class with an incompatible interface
Adapter	Wraps the Adaptee and implements the Target interface, translating calls between them
Client	Works with objects through the Target interface

2. Where and Why It Is Needed

Problem in the Codebase

The Reservation System currently has **no notification mechanism**. When a customer books a flight and completes payment, there is no way to send a booking confirmation via email or SMS. In a real-world system, notifications would be handled by **third-party services** (e.g., SendGrid for email, Twilio for SMS), each having its own proprietary API with different method signatures, parameter types, and return values.

Why the Adapter Pattern Is the Right Fit

Challenge	How the Adapter Pattern Solves It
Third-party email services use methods like `sendMail(to, subject, body, isHtml)` which return integer status codes	The Adapter translates this into our simple `sendNotification(recipient, message)` → `boolean` interface
Third-party SMS gateways use methods like `dispatchSMS(phone, text, priority)` with extra parameters	The Adapter hides the complexity and provides sensible defaults for extra parameters
Adding a new notification channel (e.g., push notifications) would require changes across the codebase	With adapters, adding a new channel means writing one new Adapter class—no existing code changes
Client code (`OrderNotifier`) would be tightly coupled to specific third-party APIs	The Adapter decouples client code, which programs only to the `NotificationService` interface

Specific Application

We introduced the following new classes in the `flight.reservation.notification` package:

Class	Role in Pattern	Description
`NotificationService`	**Target Interface**	Defines `sendNotification(recipient, message)` and `getNotificationType()`

Class	Role in Pattern	Description
`ThirdPartyEmailService`	**Adaptee**	Simulates an external email API with `sendMail(toAddress, subject, body, isHtml)` returning `int`
`ThirdPartySMSService`	**Adaptee**	Simulates an external SMS gateway with `dispatchSMS(phoneNumber, text, priority)` returning `boolean`
`EmailNotificationAdapter`	**Adapter**	Implements `NotificationService`, wraps `ThirdPartyEmailService`, translates calls
`SMSNotificationAdapter`	**Adapter**	Implements `NotificationService`, wraps `ThirdPartySMSService`, translates calls
`OrderNotifier`	**Client**	Uses `NotificationService` interface to send booking confirmations through any registered adapter

3. Benefits and Drawbacks

Benefits

- **Single Responsibility Principle**: The interface conversion logic is separated from the business logic of both the client and the third-party service.
- **Open/Closed Principle**: New notification channels (e.g., push notifications, Slack) can be added by creating new adapters without modifying existing code.
- **Loose Coupling**: `OrderNotifier` and `FlightOrder` never depend on concrete third-party implementations.
- **Testability**: Each adapter can be tested independently; the client can be tested with mock `NotificationService` implementations.
- **Flexibility**: Different adapters can be plugged in at runtime (e.g., use SMS in production, use a mock in tests).

Drawbacks

- **Additional Classes**: Each new third-party service requires a new Adapter class, which increases the number of classes in the project.
- **Indirection**: Adds a layer of indirection between the client and the actual service, which can make debugging slightly harder.
- **Translation Overhead**: Some information may be lost or defaulted during translation (e.g., the email adapter defaults `isHtml` to `false` and uses a default subject line).

4. Code Changes

New Files Created

```
src/main/java/flight/reservation/notification/
├── NotificationService.java          (Target interface)
├── ThirdPartyEmailService.java       (Adaptee - external email API)
├── ThirdPartySMSService.java         (Adaptee - external SMS API)
├── EmailNotificationAdapter.java     (Adapter for email)
├── SMSNotificationAdapter.java       (Adapter for SMS)
└── OrderNotifier.java                (Client using the adapters)

src/test/java/flight.reservation/
└── NotificationAdapterTest.java      (14 unit tests)
```

Key Code Snippets

Target Interface – `NotificationService`

```java
public interface NotificationService {
    boolean sendNotification(String recipient, String message);
    String getNotificationType();
}
```

Adaptee – `ThirdPartyEmailService` (incompatible interface)

```
public class ThirdPartyEmailService {
    // Returns int status code, takes 4 parameters — incompatible with NotificationService
    public int sendMail(String toAddress, String subject, String body, boolean isHtml) { ... }
}
```

Adapter – `EmailNotificationAdapter`

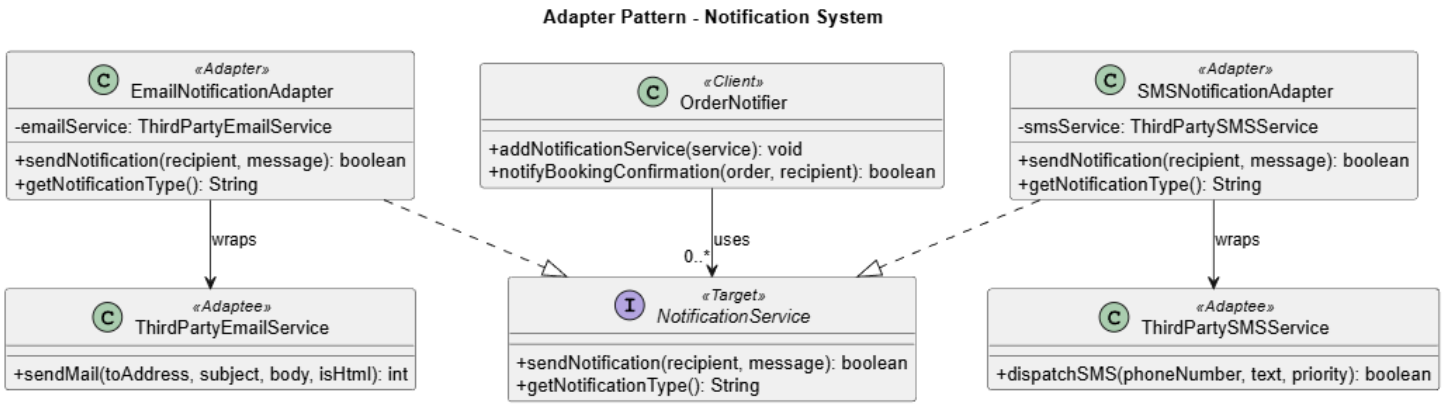
```
public class EmailNotificationAdapter implements NotificationService {
    private final ThirdPartyEmailService emailService;
    private final String defaultSubject;

    @Override
    public boolean sendNotification(String recipient, String message) {
        // Translates simple interface → complex third-party interface
        int statusCode = emailService.sendMail(recipient, defaultSubject, message, false);
        return statusCode == 0;
    }
}
```

Client Usage – `OrderNotifier`

```
// Client uses only the NotificationService interface — fully decoupled
OrderNotifier notifier = new OrderNotifier();
notifier.addNotificationService(new EmailNotificationAdapter(new ThirdPartyEmailService()));
notifier.addNotificationService(new SMSNotificationAdapter(new ThirdPartySMSService()));
notifier.notifyBookingConfirmation(order, "customer@email.com");
```

5. Class Diagram – After Applying the Adapter Pattern

Since this is a **new feature** (no notification system existed before), only the "after" diagram is provided.



How to Read the Diagram

Relationship	Meaning
`EmailNotificationAdapter ..|> NotificationService`	Adapter **implements** the Target interface
`EmailNotificationAdapter --> ThirdPartyEmailService`	Adapter **wraps** (composes) the Adaptee
`OrderNotifier --> NotificationService`	Client **depends on** the Target interface only
`OrderNotifier ..> FlightOrder`	Client **uses** FlightOrder to build notification messages

Pattern Flow

```
OrderNotifier (Client)
    │
    │ calls sendNotification(recipient, message)
    ▼
NotificationService (Target Interface)
    │
    ├──▶ EmailNotificationAdapter (Adapter)
    │        │
    │        │ translates to sendMail(to, subject, body, isHtml)
    │        ▼
    │    ThirdPartyEmailService (Adaptee)
    │
    └──▶ SMSNotificationAdapter (Adapter)
             │
             │ translates to dispatchSMS(phone, text, priority)
             ▼
         ThirdPartySMSService (Adaptee)
```

6. Assumptions

1. The `ThirdPartyEmailService` and `ThirdPartySMSService` classes simulate external third-party libraries whose source code cannot be modified (a core precondition for needing the Adapter pattern).
2. The email adapter uses a default subject line ("Flight Reservation Notification") and sends plain-text emails (`isHtml = false`).
3. The SMS adapter defaults to normal priority (`priority = 2`).
4. `OrderNotifier` sends notifications through **all** registered services — this enables multi-channel notifications.
5. The notification feature is a new addition to the codebase; no existing classes were modified.